

Series: STIG Hardening Docker Containers & Applications (Part 5 of 6)

Author: H. Brister Wooley

Published: 2026-05-23

Series: STIG Hardening Docker Containers & Applications (5/6)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-05-23-stig-hardening-docker-part5.md>

Welcome to Part 5.

We've hardened the base image, the Dockerfile, and the runtime. The container is a fortress. But the application *inside* the container? That's where 60% of STIG findings live.

Container settings can't fix SQL injection. They can't enforce input validation. They can't make your session tokens secure. They can't implement audit logging. Those are **application-layer controls** – and they're the ones that fail audits most often.

We'll use **real V-IDs from your project's Application Server STIG** (V-204708 through V-204834, plus V-35xxx, V-46xxx, V-57xxx series) and cross-reference with Apache STIG V-214256.

The Application STIG Controls Map (Real V-IDs from Your Project)

STIG Control	V-ID	Severity	STIG ID	What It Requires	Where It Lives
Concurrent Session Limit	V-204708	Medium	SRG-APP-000001	Limit concurrent sessions per account/type	App code
Session Logging on Startup	V-204720	Medium	SRG-APP-000092	Initiate session logging at startup	App code + infra
Input Validation	V-214256	Medium	AS24-U1-000620	Validate all inputs (type, length, format, range)	App code
Output Encoding	V-214256	Medium	AS24-U1-000620	Encode for context (HTML, JS, URL, SQL, Shell)	App code

STIG Control	V-ID	Severity	STIG ID	What It Requires	Where It Lives
SQL Injection Prevention	V-214256	Medium	AS24-U1-000620	Parameterized queries, no dynamic SQL	App code
XSS Prevention	V-214256	Medium	AS24-U1-000620	CSP, output encoding, input validation	App code + headers
Session Management	V-214256	Medium	AS24-U1-000620	Secure, HttpOnly, SameSite, rotation, timeout	App code
Authentication	V-204709+	Medium	SRG-APP-000014+	MFA, password policy, account lockout	App code + infra
Authorization	V-214256	Medium	AS24-U1-000620	RBAC, least privilege, object-level checks	App code
Cryptographic Standards	V-214256, V-35xx x	Medium	AS24-U1-000620	AES-256, TLS 1.2+, SHA-256+, no MD5/SHA1	App code + TLS
Audit Logging	V-204720, V-214231	Medium	SRG-APP-000092, AS24-U1-000040	Who, what, when, where, outcome	App code
Error Handling	V-214256	Medium	AS24-U1-000620	No stack traces, no info leakage, generic messages	App code
Session ID Generation	V-204709+	Medium	SRG-APP-000014+	Long, random, TLS-only, invalidated on logout	App code
Account Lockout	V-204709+	Medium	SRG-APP-000014+	Lockout after failed attempts	App code
Password Complexity	V-204709+	Medium	SRG-APP-000014+	Min length, complexity, history, expiration	App code

Input Validation: The allowlist Pattern (V-214256 / AS24-U1-000620)

```
# BAD: Denylist (always incomplete)
def validate_username(username):
    banned = ["'", "\"", ";", "--", "/*", "*/", "xp_", "sp_"]
    for b in banned:
        if b in username:
            raise ValueError("Invalid input")
    return username

# GOOD: Allowlist (explicit is safe)
import re
from pydantic import BaseModel, field_validator

class UserCreate(BaseModel):
    username: str
    email: str
    role: str

    @field_validator("username")
    @classmethod
    def validate_username(cls, v: str) -> str:
        # Only alphanumeric, underscore, hyphen, 3-32 chars
        if not re.fullmatch(r"[a-zA-Z0-9_-]{3,32}", v):
            raise ValueError("Username must be 3-32
alphanumeric/underscore/hyphen")
        return v

    @field_validator("email")
    @classmethod
    def validate_email(cls, v: str) -> str:
        # RFC 5322 compliant (simplified)
        if not re.fullmatch(r"^[^@]+@[^@]+\.[^@]+$", v):
            raise ValueError("Invalid email format")
        return v

    @field_validator("role")
    @classmethod
    def validate_role(cls, v: str) -> str:
        allowed = ["user", "admin", "analyst", "viewer"]
        if v not in allowed:
            raise ValueError(f"Role must be one of: {allowed}")
        return v
```

STIG V-214256 (AS24-U1-000620): “The application must validate all input for type, length, format, and range.”

V-204708 (SRG-APP-000001): Session limits – apply rate limiting to validation endpoints too.

Verification: Unit tests for every validator – valid inputs pass, invalid inputs fail with 400.
Load test: 1000 concurrent validation requests -> no DoS.

Output Encoding: Context Matters (V-214256 / AS24-U1-000620)

BAD: One encoding for everything

```
def render(user_input):  
    return f"<div>{user_input}</div>" # XSS if input = <script>...
```

GOOD: Context-specific encoding

```
from markupsafe import escape  
import html  
import urllib.parse
```

```
def render_html(user_input: str) -> str:  
    """HTML body context"""  
    return f"<div>{escape(user_input)}</div>"
```

```
def render_js(user_input: str) -> str:  
    """JavaScript string context"""  
    # Escape for JS string: \, ', ", \n, \r, \t, </  
    escaped = user_input.replace("\\", "\\\").replace("'", "\\'").replace("\n", "\\n").replace("\r", "\\r").replace("\t", "\\t").replace("</", "<\\/")  
    return f"const userInput = '{escaped}';"
```

```
def render_url(user_input: str) -> str:  
    """URL query parameter context"""  
    return urllib.parse.quote(user_input, safe="")
```

```
def render_sql(user_input: str) -> str:  
    """SQL context -- NEVER DO THIS. Use parameterized queries instead."""  
    raise NotImplementedError("Use parameterized queries, not escaping")
```

STIG V-214256 (AS24-U1-000620): “The application must encode output for the appropriate context.”

Why context matters: HTML encoding (<) does nothing in a JavaScript string. URL encoding (%3C) does nothing in HTML. Each context has its own rules.

Verification: `grep -r "f\".*<|f\".*script" --include="*.py" --include="*.js" --include="*.tsx" -> should return nothing.`

SQL Injection: Parameterized Queries Only (V-214256 / AS24-U1-000620)

BAD: String interpolation

```
async def get_user_bad(db, username: str):
    query = f"SELECT * FROM users WHERE username = '{username}'"
    return await db.fetch(query)
```

GOOD: Parameterized query

```
async def get_user_good(db, username: str):
    query = "SELECT * FROM users WHERE username = $1"
    return await db.fetch(query, username)
```

GOOD: ORM (SQLAlchemy example)

```
async def get_user_orm(db, username: str):
    return await db.execute(select(User).where(User.username == username))
```

STIG V-214256 (AS24-U1-000620): “The application must use parameterized queries for all database interactions.”

No exceptions. Not for “simple” queries. Not for “internal” tools. Not for “trusted” inputs. Every. Single. Query.

Verification: `grep -r "f\".*SELECT\\|f\".*INSERT\\|f\".*UPDATE\\|f\".*DELETE" --include="*.py" ->` should return nothing.

Session Management: The Secure Cookie Pattern (V-204709+, V-214256)

```
from fastapi import Response
from datetime import timedelta
import secrets
import json
from datetime import datetime
```

```
SESSION_COOKIE_NAME = "session"
SESSION_MAX_AGE = timedelta(minutes=30).total_seconds()
```

V-204708 (SRG-APP-000001): Concurrent session limit

```
MAX_CONCURRENT_SESSIONS = 5
```

```
async def create_session(response: Response, user_id: str, roles: list[str],
redis) -> str:
    # V-204708: Check concurrent sessions
    current_sessions = await redis.scard(f"user_sessions:{user_id}")
    if current_sessions >= MAX_CONCURRENT_SESSIONS:
        raise HTTPException(status_code=429, detail="Maximum concurrent
```

```

sessions_reached")

    session_id = secrets.token_urlsafe(32)

    # Store in Redis with TTL
    session_data = {
        "user_id": user_id,
        "roles": roles,
        "created": datetime.utcnow().isoformat(),
        "csrf_token": secrets.token_urlsafe(32)
    }
    await redis.setex(f"session:{session_id}", int(SESSION_MAX_AGE),
json.dumps(session_data))

    # V-204708: Track session for concurrent limit
    await redis.sadd(f"user_sessions:{user_id}", session_id)
    await redis.expire(f"user_sessions:{user_id}", int(SESSION_MAX_AGE))

    # Set secure cookie
    response.set_cookie(
        key=SESSION_COOKIE_NAME,
        value=session_id,
        max_age=int(SESSION_MAX_AGE),
        httponly=True,          # No JavaScript access (V-214256)
        secure=True,            # HTTPS only (V-214256)
        samesite="strict",      # No cross-site sends (V-214256)
        path="/"
    )

    return session_id

async def destroy_session(response: Response, session_id: str, user_id: str,
redis):
    # V-204708: Clean up session tracking
    await redis.srem(f"user_sessions:{user_id}", session_id)
    await redis.delete(f"session:{session_id}")
    response.delete_cookie(SESSION_COOKIE_NAME, path="/")

```

Cookie Attribute	STIG V-ID	Why
HttpOnly	V-214256	Prevents XSS theft
Secure	V-214256	HTTPS only – no plaintext transmissi on
SameSite=Strict	V-214256	CSRF

Cookie Attribute	STIG V-ID	Why
		protection – no cross-origin sends
Max-Age	V-214256	Session timeout (30 min default)
Path=/ 	V-214256	Scoped to application root
Concurrent limit	V-204708	DoS prevention

STIG V-214256: “Session IDs must be long, random, transmitted only over TLS, and invalidated on logout/timeout.”

Cryptographic Standards: The Approved List (V-214256, V-35xxx)

APPROVED algorithms (STIG V-214256, V-35301, V-35302, V-35304)

```
CRYPTO_STANDARDS = {
    "symmetric_encryption": "AES-256-GCM",
    "asymmetric_encryption": "RSA-4096 / ECDSA-P384",
    "key_derivation": "PBKDF2 (100k+ iterations) / Argon2id",
    "hashing": "SHA-256 / SHA-384 / SHA-512",
    "mac": "HMAC-SHA-256 / AES-GCM tag",
    "tls": "TLS 1.2+ with AES-256-GCM / CHACHA20-POLY1305",
    "certificates": "RSA-4096 / ECDSA-P384, SHA-256+"
}
```

BANNED algorithms (will cause CAT I/II findings)

```
BANNED = {
    "MD5", "SHA1", "DES", "3DES", "RC4", "AES-ECB", "AES-CBC",
    "RSA-1024", "DSA-1024", "DH-1024", "EC-DSA-P256"
}
```

Usage example

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
```

```
def encrypt_data(plaintext: bytes, key: bytes) -> tuple[bytes, bytes]:
    """AES-256-GCM encryption with random nonce."""
    aesgcm = AESGCM(key)
    nonce = secrets.token_bytes(12) # 96-bit nonce for GCM
```

```
ciphertext = aesgcm.encrypt(nonce, plaintext, None)
return nonce, ciphertext
```

Verification: `grep -r "md5\|sha1\|des\|rc4\|ecb\|cbc" --include="*.py" --include="*.js" --include="*.go" ->` should only appear in comments or tests.

Audit Logging: The Structured Event Pattern (V-204720, V-214231)

```
import structlog
from datetime import datetime
from enum import Enum
from typing import Optional, Dict, Any

class AuditEventType(Enum):
    LOGIN_SUCCESS = "auth.login.success"
    LOGIN_FAILURE = "auth.login.failure"
    LOGOUT = "auth.logout"
    PASSWORD_CHANGE = "auth.password.change"
    PERMISSION_DENIED = "authz.denied"
    DATA_ACCESS = "data.access"
    DATA_MODIFY = "data.modify"
    DATA_DELETE = "data.delete"
    ADMIN_ACTION = "admin.action"
    CONFIG_CHANGE = "config.change"

# V-204720 (SRG-APP-000092): Session Logging on startup
def log_startup():
    logger = structlog.get_logger()
    logger.info("audit", **{
        "event_type": "system.startup",
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "outcome": "success",
        "details": {"version": "1.0.0", "config_hash": "abc123"}
    })

logger = structlog.get_logger()

def audit_log(
    event_type: AuditEventType,
    user_id: str,
    resource: Optional[str] = None,
    action: Optional[str] = None,
    outcome: str = "success",
    details: Optional[Dict[str, Any]] = None,
    request=None
):
    """Structured audit log entry -- V-204720, V-214231 compliant."""
    log_data = {
```

```

        "event_type": event_type.value,
        "user_id": user_id,
        "timestamp": datetime.utcnow().isoformat() + "Z",
        "outcome": outcome,
        "source_ip": request.client.host if request else None,
        "user_agent": request.headers.get("user-agent") if request else None,
        "resource": resource,
        "action": action,
        "details": details or {}
    }

    logger.info("audit", **log_data)
    # Also send to SIEM (Elastic, Splunk, etc.)
    # siem_client.send(log_data)

```

Example log entry (V-204720, V-214231 compliant):

```

{
    "event_type": "data.access",
    "user_id": "user_12345",
    "timestamp": "2026-09-23T14:32:15Z",
    "outcome": "success",
    "source_ip": "192.0.2.100",
    "user_agent": "Mozilla/5.0...",
    "resource": "document",
    "action": "read",
    "details": {"document_id": "doc_98765", "classification": "CUI"}
}

```

STIG V-204720 (SRG-APP-000092): “The application server must initiate session logging upon startup.” **STIG V-214231 (AS24-U1-000040):** “Audit records must contain: date/time, user ID, event type, outcome, source IP, resource affected.”

The One Finding That Fails 90% of Audits

V-214256 (AS24-U1-000620): “The application must not expose detailed error messages to users.”

```

# BAD: Stack trace to user
@app.exception_handler(Exception)
async def bad_handler(request, exc):
    return JsonResponse(
        status_code=500,
        content={"detail": str(exc), "traceback": traceback.format_exc()}
    )

```

```

# GOOD: Generic message, detailed Log
@app.exception_handler(Exception)

```

```

async def good_handler(request, exc):
    # Log the full details server-side
    logger.error("unhandled_exception",
        path=request.url.path,
        method=request.method,
        user_id=getattr(request.state, "user_id", None),
        error=str(exc),
        traceback=traceback.format_exc()
    )

    # Return generic message to user
    return JsonResponse(
        status_code=500,
        content={"detail": "An internal error occurred. Reference ID: abc-123-def"}
    )

```

Why it fails: Developers leave debug handlers in production. Framework defaults (DEBUG=True) expose stack traces. Error pages leak framework versions, library paths, SQL snippets.

Fix in 10 lines: Global exception handler that logs everything, returns nothing.

Account Lockout & Password Policy (V-204709+ / SRG-APP-000014+)

```

from passlib.context import CryptContext

```

```

pwd_context = CryptContext(schemes=["argon2"], deprecated="auto")

```

```

# V-204709+: Password complexity

```

```

PASSWORD_POLICY = {
    "min_length": 15,
    "require_uppercase": True,
    "require_lowercase": True,
    "require_digits": True,
    "require_special": True,
    "max_age_days": 90,
    "history_count": 24
}

```

```

# V-204709+: Account Lockout

```

```

MAX_FAILED_ATTEMPTS = 5
LOCKOUT_DURATION = timedelta(minutes=30)

```

```

async def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)

```

```

def hash_password(plain: str) -> str:

```

```

    return pwd_context.hash(plain)

# In Login endpoint:
async def login(username: str, password: str, redis):
    # Check Lockout
    failed_key = f"login_failures:{username}"
    attempts = await redis.get(failed_key)
    if attempts and int(attempts) >= MAX_FAILED_ATTEMPTS:
        raise HTTPException(429, "Account locked. Try again in 30 minutes.")

    user = await get_user(username)
    if not user or not await verify_password(password, user.hashed_password):
        # Increment failure counter
        await redis.incr(failed_key)
        await redis.expire(failed_key, int(LOCKOUT_DURATION.total_seconds()))
        raise HTTPException(401, "Invalid credentials")

    # Success -- reset failure counter
    await redis.delete(failed_key)
    return await create_session(...)

```

What This Means for Your Strategy

Application-layer STIG is where **developers** own the controls. Not DevOps. Not platform. The code must be right.

Your strategy: 1. **Codify standards** – the allowlist validators, encoding functions, parameterized query patterns, audit log schema – as shared libraries 2. **Enforce in CI** – bandit, semgrep, custom rules for your patterns 3. **Test in pipeline** – security unit tests, SAST, DAST in staging 4. **Document the decisions** – why AES-256-GCM, why 30-min sessions, why Strict SameSite

The auditor doesn't check your code. They check your *running application*. But the only way the running app passes is if the code is right.

What's Coming Next

Part 6: Verification, automation, and the CI/CD pipeline that generates the evidence package. The nightly compliance scan. The STIG update workflow. The audit-ready dashboard.

Plus the one script that generates your entire STIG evidence package in 3 minutes.

Read Part 6 ->

Appendix: CompTIA Security+ SY0-701 CEU Mapping

This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Allowlist input validation (Pydantic)	Domain 2: Security Architecture	2.3: Implement application security	Explicit validation prevents injection attacks
Context-specific output encoding	Domain 2: Security Architecture	2.3: Implement application security	XSS prevention via proper escaping per context
Parameterized queries (no string interpolation)	Domain 2: Security Architecture	2.3: Implement application security	SQL injection prevention
Secure session cookies (HttpOnly, Secure, SameSite=Strict)	Domain 2: Security Architecture	2.3: Implement application security	Session hijacking, CSRF prevention
Concurrent session limiting	Domain 1: Threats/Vulns/Mitigations	1.4: Analyze indicators of malicious activity	DoS prevention, credential stuffing mitigation
Argon2id password hashing + complexity policy	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Strong credential storage
Account lockout after failed attempts	Domain 1: Threats/Vulns/Mitigations	1.4: Analyze indicators of malicious activity	Brute force prevention
Approved crypto standards (AES-256-GCM, SHA-256+, TLS 1.2+)	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Cryptographic algorithm validation
Structured audit logging (who, what, when, where,	Domain 4: Security Operations	4.1: Apply security monitoring concepts	Audit trail for incident

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
outcome)			response
Generic error messages + detailed server-side logging	Domain 2: Security Architecture	2.3: Implement application security	Information leakage prevention
Startup session logging	Domain 4: Security Operations	4.1: Apply security monitoring concepts	System lifecycle auditing (V-204720)

Primary Domain Coverage: Domain 2 (Architecture) – ~40% | Domain 3 (Architecture & Engineering) – ~25% | Domain 1 (Threats) – ~20% | Domain 4 (Operations) – ~15%

Total Security+ Mapping: >=50% of content maps to SY0-701 objectives